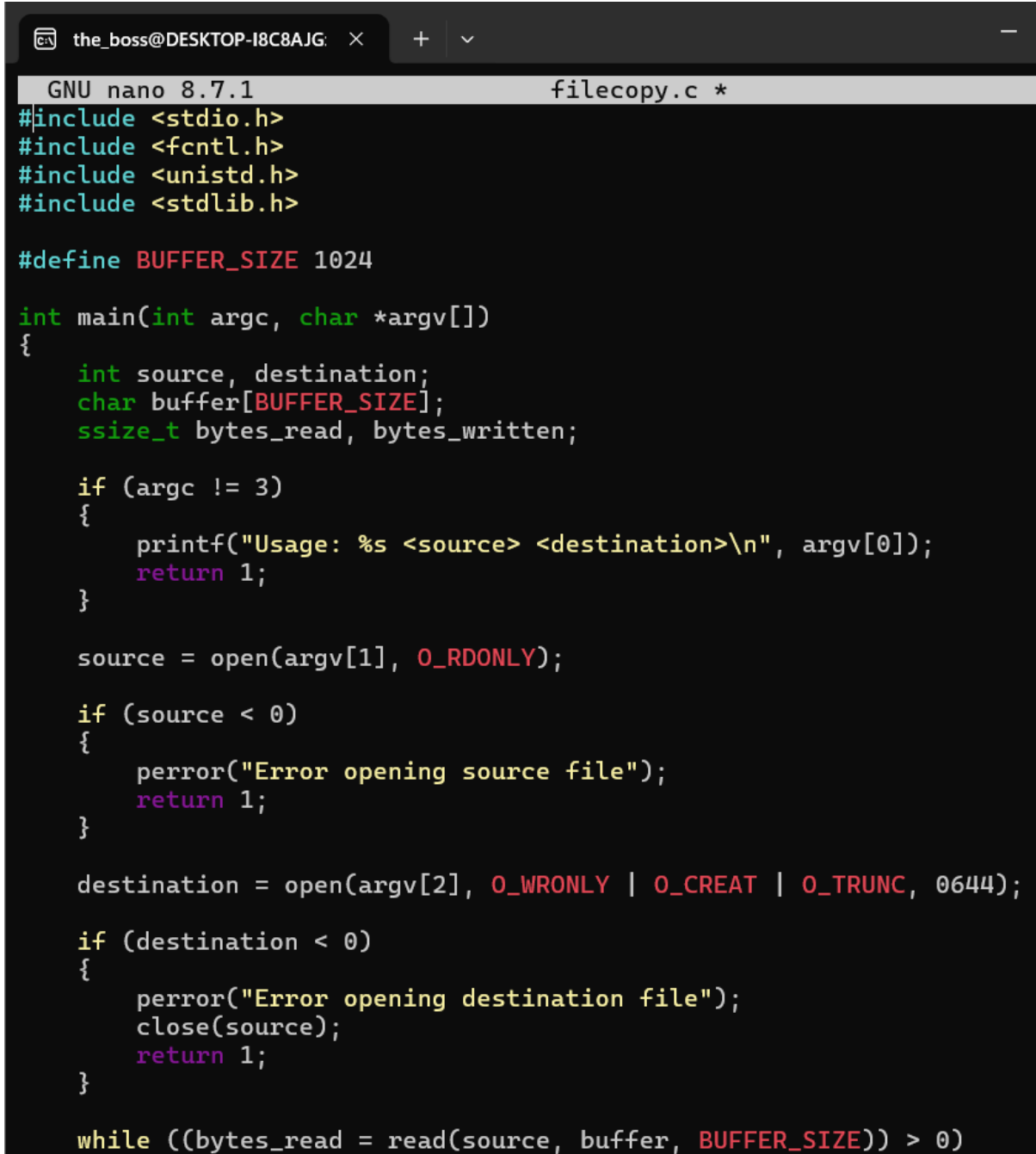


WEEK-2 Practical

TASK-1 Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user Space and kernel space during execution.



```
GNU nano 8.7.1 filecopy.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

#define BUFFER_SIZE 1024

int main(int argc, char *argv[])
{
    int source, destination;
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read, bytes_written;

    if (argc != 3)
    {
        printf("Usage: %s <source> <destination>\n", argv[0]);
        return 1;
    }

    source = open(argv[1], O_RDONLY);

    if (source < 0)
    {
        perror("Error opening source file");
        return 1;
    }

    destination = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (destination < 0)
    {
        perror("Error opening destination file");
        close(source);
        return 1;
    }

    while ((bytes_read = read(source, buffer, BUFFER_SIZE)) > 0)
```

Output:

```
the_boss@DESKTOP-I8C8AJG:~/ossp$ nano filecopy.c
the_boss@DESKTOP-I8C8AJG:~/ossp$ gcc -Wall -Wextra filecopy.c -o bin/filecopy
y
the_boss@DESKTOP-I8C8AJG:~/ossp$ echo "This is a sample file for system call
testing." > sample.txt
the_boss@DESKTOP-I8C8AJG:~/ossp$ ./bin/filecopy sample.txt copy.txt
File copied successfully.
the_boss@DESKTOP-I8C8AJG:~/ossp$ cat sample.txt
cat copy.txt
This is a sample file for system call testing.
This is a sample file for system call testing.
```

Report: I developed a C program to copy the contents of one file to another using the system calls `open()`, `read()`, `write()`, and `close()`. The source file was opened in read-only mode and the destination file was created for writing. The contents were read into a buffer and written to the destination file. During execution, the system calls transfer control from user space to kernel space, where the kernel performs the required file operations and returns the results to the program. The file was copied successfully.

Task-2 : Use the `strace` utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system 3 calls and identify the kernel services involved

```

the_boss@DESKTOP-I8C8AJG: ~ /oss$ strace -e trace=openat,read,write,close cat sample.txt
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
close(3) = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libselinux.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0"..., 832) = 832
close(3) = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libgcc_s.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0"..., 832) = 832
close(3) = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libm.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0"..., 832) = 832
close(3) = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0 \250\2\0\0\0\0\0"..., 832) = 832
close(3) = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libpcre2-8.so.0", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0"..., 832) = 832
close(3) = 0
openat(AT_FDCWD, "/proc/filesystems", O_RDONLY|O_CLOEXEC) = 3
read(3, "nodev\tsysfs\nnodev\ttmpfs\nnodev\tbd"..., 1024) = 442
close(3) = 0
openat(AT_FDCWD, "/proc/mounts", O_RDONLY|O_CLOEXEC) = 3
read(3, "none /usr/lib/modules/6.18.33.2-"..., 1024) = 1024
read(3, "e 0 0\ndepts /dev/pts devpts rw,"..., 1024) = 1024
read(3, ",access=client,msize=65536,trans"..., 1024) = 1024
read(3, "slg/run/user/1000 tmpfs rw,nosui"..., 1024) = 396
read(3, "", 1024) = 0
close(3) = 0
openat(AT_FDCWD, "/proc/self/maps", O_RDONLY|O_CLOEXEC) = 3
read(3, "59d8638b3000-59d8639a5000 r--p 0"..., 1024) = 1024

```

Report:

I used the `strace` utility to trace the system calls generated by the command `cat sample.txt`. The trace showed important system calls such as `openat()`, `read()`, `write()`, and `close()`. `openat()` opens the file, `read()` obtains the file contents, `write()` displays the contents on the terminal, and `close()` closes the file descriptor. The experiment helped me understand how a user-level command interacts with kernel services through system calls.